OpsMind AI: Enterprise Standard Operating Procedure (SOP) Knowledge Assistant with Retrieval-Augmented Generation

A report submitted for the course named Internship (CS4204)

Submitted By

Krishna Anand

SEMESTER - VII

230101082

Supervised By

Dr. Rajkumari Bidyalakshmi Devi



Department of Computer Science and Engineering

Indian Institute of Information Technology Senapati, Manipur

September, 2026

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

KRISHNA ANAND
230101082

Date: 22-September-2026

Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. Rajkumari Bidyalakshmi Devi
Assistant Professor, CSE, IIIT Manipur

Email: bidyalakshmi@iiitmanipur.ac.in
Contact No: +91-7005126046

To Whom It May Concern

This is to certify that the internship report entitled **"OpsMind AI: Enterprise Standard Operating Procedure (SOP) Knowledge Assistant with Retrieval-Augmented Generation",** submitted to the Department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fulfillment for the award of degree of **Bachelor of Technology in Computer Science and Engineering** is a record of bonafide work carried out by **Krishna Anand** bearing roll number **230101082**.

Signature of Faculty Advisor

(Dr. Rajkumari Bidyalakshmi Devi)

Signature of the Examiner 1: .....

Signature of the Examiner 2: .....

Signature of the Examiner 3: .....

Signature of the Examiner 4: .....

Offer Letter

Internship Offer Letter

ZAALIMA DEVELOPMENT

20 March 2026

Zaalima technologies
Electronic City in Bangalore, Karnataka 560100

Dear Krishna Anand,

We are pleased to offer you an internship position as a Web Developer at Zaalima Development Pvt.Ltd. After careful consideration of your application and interview, we believe that your skills and enthusiasm align well with our internship program.

Internship Details:

- Position: Web Developer
- Duration: 25-03-2026 –25-06-2026
- Weekly Hours: Flexible working hours
- Location: Remote
- Stipend: Up to ₹6,000 per month, based on performance evaluation.

Your internship will provide you with valuable industry exposure, hands-on experience, and an opportunity to contribute to our projects. The stipend will be performance-based, and evaluations will be conducted periodically based on your work quality, adherence to deadlines, and overall contribution.

Rules and Regulations:

- Confidentiality: You must maintain the confidentiality of all company information and data during and after the internship.
- Work Ethics: Professional behavior, punctuality, and a proactive attitude are expected throughout the internship.
- Performance Evaluation: Your stipend will depend on your performance, assessed through task completion, dedication, and teamwork.

ZAALIMA TEChNOLOgY

ZAALIMA DEVELOPMENT

www.zaalima.in
+91 8277035909
support@zaalima.in

Offer Letter

ZAALIMA DEVELOPMENT

- Code of Conduct: You are required to follow all workplace policies and guidelines provided by your supervisor.
- Leave Policy: Interns must inform their supervisors in advance if they need any leave. Unapproved absences may impact stipend eligibility.
- Completion Certificate: A certificate of completion will be provided upon successful fulfillment of the internship requirements.
- Termination Clause: The company reserves the right to terminate the internship in case of misconduct, poor performance, or violation of company policies.

To accept this offer, please sign and return a scanned copy of this letter by [Acceptance Deadline].

We look forward to having you on board and witnessing your valuable contributions to our team.

Acceptance Acknowledgment

I, Krishna Anand, accept the internship offer and agree to the terms and conditions mentioned above.

Signature: Krishna Anand
Date: 21/03/26

ZAALIMA DEVELOPMENT PRIVATE LIMITED

Sincerely,

Mr. Padmalabha Gouda

ZAALIMA TEChNOLOgY

Completion Certificate



Abstract

In many companies, information is scattered across different documents and departments, making it hard for employees to find what they need quickly. During my internship, I worked on OpsMind AI, a tool that solves this problem. It is a web application built with React, Node.js, and MongoDB that lets users search through company documents using natural language. Under the hood, it uses Retrieval-Augmented Generation (RAG) by combining MongoDB Atlas Vector Search with the Gemini API. The system parses PDF documents, breaks them into chunks, and stores their vector embeddings. When a user asks a question, the system retrieves relevant information from the indexed SOP documents and generates a context-aware answer using the Gemini API. This project helped me learn how to build practical AI features into a full-stack application, ultimately making information retrieval much faster for corporate teams.

Acknowledgement

I would like to express my sincere gratitude to my industry supervisor, **Mr. Santanu Badatya** (Project Manager & Tech Lead), and the entire team at **Zaalima Development** for giving me the opportunity to work as an intern. My time there provided invaluable hands-on experience, and their mentorship was a huge help while I was building DravixAi (OpsMind AI). I am also highly thankful to my fellow team members and interns for their valuable discussions and feedback, which made a significant difference in bringing this system to completion.

Furthermore, I am profoundly grateful to my academic project supervisor, **Dr. Rajkumari Bidyalakshmi Devi**, for her constant guidance and insightful direction. Her expertise and constructive feedback were instrumental in shaping the core architecture of this project. I would also like to thank the faculty members and my peers at **IIIT Manipur** for their continuous support and valuable discussions during the development and testing phases. Finally, I would like to thank my friends for their unwavering encouragement throughout this journey.

Krishna Anand

Contents

Declaration i

Certificate ii

Offer Letter (Page 1) iii

Offer Letter (Page 2) iv

Completion Certificate v

Abstract vi

Acknowledgement vii

List of Figures x

List of Tables xi

1 Introduction 1
- 1.1 Outline . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Motivation . . . 2
- 1.4 Purpose . . . 2

2 Literature Survey 4
- 2.1 Evolution of Enterprise Information Retrieval . . . 4
  - 2.1.1 Lexical and Probabilistic Retrieval Models . . . 4
  - 2.1.2 Dense Vector Semantic Representations . . . 4
- 2.2 Transformer Distillation and Compact Embeddings . . . 5
- 2.3 Retrieval-Augmented Generation (RAG) Framework . . . 5
- 2.4 Vector Databases and Approximate Nearest Neighbor Search . . . 6
- 2.5 Large Language Models and Grounding Techniques . . . 6

3 Requirement Engineering 7
- 3.1 Functional Requirements . . . 7
- 3.2 Non-Functional Requirements . . . 9
- 3.3 Hardware and Software Requirements . . . 9
- 3.4 Use Case Diagram . . . 10
  - 3.4.1 Actors . . . 10
  - 3.4.2 Use Case Model . . . 10

4 System Design **12**
4.1 System Architecture . . . 12
4.2 Data Ingestion Flowchart . . . 13
4.3 UML Component Diagram . . . 15
4.4 Database Schema and Vector Index Design . . . 15
4.4.1 Chunk Collection Data Model . . . 15
4.4.2 Vector Search Index Specification . . . 16
4.5 Text Segmentation and Domain Classification . . . 16
4.5.1 Sliding-Window Segmentation . . . 17
4.5.2 Domain Section Classification . . . 17
4.6 API Endpoint Structure . . . 17

5 Implementation **19**
5.1 Document Processing Pipeline Implementation . . . 19
5.1.1 PDF Text Stream Ingestion . . . 19
5.1.2 Sliding-Window Text Segmentation . . . 19
5.2 Vector Search Pipeline and Aggregation Engine . . . 20
5.2.1 Retrieval Relevance and Fallback Logic . . . 20
5.3 Large Language Model Integration . . . 20
5.4 Administrative Management Subsystem . . . 20
5.5 Frontend Architecture and Component Engineering . . . 21
5.6 Error Handling and System Edge Cases . . . 21

6 Results **23**
6.1 Experimental Setup and Evaluation Corpus . . . 23
6.2 Document Ingestion and Analytics Verification . . . 23
6.3 Conversational Retrieval and Attribution Verification . . . 24
6.3.1 Multi-Step Operational Governance Query Evaluation . . . 25
6.4 Qualitative and Quantitative Test Case Analysis . . . 26
6.5 Latency Breakdown and Performance Analysis . . . 26

7 Conclusion **27**
7.1 Summary of Work and Key Contributions . . . 27
7.2 Limitations of the Current System . . . 28
7.3 Future Scope and Enhancements . . . 28

Bibliography **30**

List of Figures

3.1 UML Use Case Diagram of OpsMind AI 11

4.1 Tiered System Architecture of OpsMind AI 13
4.2 Data Ingestion Flowchart of OpsMind AI 14
4.3 UML Component Diagram of OpsMind AI 15

6.1 OpsMind AI Document Ingestion Interface and Analytics Dashboard 24
6.2 OpsMind AI Conversational Assistant Initial Interface 24
6.3 End-to-End Query Answering and Granular Source Citation Inspection 25

List of Tables

3.1 System Hardware and Software Specifications 10

4.1 Database Field Definitions for the Chunk Entity 16
4.2 Vector Search Index Configuration 16
4.3 OpsMind AI RESTful API Endpoint Specifications 18

6.1 Structural Metrics of the Evaluation SOP Corpus 23
6.2 Benchmark Query Execution and Evaluation Matrix 26
6.3 End-to-End Latency Profile of the OpsMind AI Pipeline 26

Chapter 1

Introduction

1.1 Outline

In large organizations, important operational information is stored in Standard Operating Procedures (SOPs), which can become difficult to search as the number and size of documents increase. Traditional keyword-based search often fails to understand the meaning of user questions, while using LLMs directly can produce incorrect answers and cannot reliably provide sources. To solve these problems, this internship developed **OpsMind AI**, an intelligent SOP Knowledge Assistant using **Retrieval-Augmented Generation (RAG)**. The system reads PDF documents, divides them into smaller overlapping sections, converts them into vector representations, and stores them in a cloud-based vector database for fast semantic search. When a user asks a question, the system finds the most relevant information and uses **Google Gemini** to generate an answer based only on the retrieved content. It also provides the document name, page number, section, chunk ID, and similarity score so that users can easily verify the answer.

1.2 Problem Statement

Modern enterprises depend on large collections of Standard Operating Procedures (SOPs) to define operational processes, approval workflows, compliance requirements, troubleshooting procedures, security policies, and other organizational guidelines. However, as these documents grow in volume and complexity, employees may spend significant time manually searching through lengthy PDFs to find specific information. Traditional keyword-based search often fails to understand the context and intent behind natural-language questions.

OpsMind AI addresses this problem by providing an AI-powered knowledge assistant that uses a **Retrieval-Augmented Generation (RAG)** approach. SOP documents are processed into searchable chunks and embeddings, allowing the system to retrieve relevant information when a user asks a question and generate a concise answer based on the retrieved content. The system also provides source references such as document, page, and section, while avoiding unsupported answers when the required information is not available in the uploaded SOPs.

1.3 Motivation

The primary motivation behind OpsMind AI is to bridge the operational gap between static enterprise documentation and agile human decision-making. By converting static enterprise documentation into an interactive semantic knowledge base, organizations can improve access to critical operational knowledge while maintaining source traceability.

Modern fulfillment centers, delivery hubs, and administrative divisions require sub-second access to operational parameters. When an operator queries an approval chain, a cycle count frequency, or an emergency evacuation protocol, the assistant must return not only the definitive procedure but also the exact page reference for institutional accountability.

Furthermore, building an end-to-end decoupled architecture—integrating in-process dense embedding pipelines, scalable vector database indexing, and advanced generative reasoning—provides a cost-effective, maintainable blueprint for next-generation enterprise knowledge management without requiring massive model retraining or specialized hardware clusters.

1.4 Purpose

The purpose of this internship work is to design, implement, and rigorously evaluate an end-to-end enterprise knowledge retrieval and conversational assistant system. Specifically, the internship aims to fulfill the following objectives:

- **Automated Document Ingestion and Segmentation:** Develop an automated ingestion pipeline capable of parsing multi-page enterprise PDF documents, executing intelligent text extraction, and generating normalized, overlapping text chunks to preserve local context across boundaries.
- **Domain-Specific Section Categorization:** Implement an automated heuristic classification system that identifies operational sections (e.g., Inbound Operations, Inventory Audit Framework, Outbound Operations, Safety, Business Continuity) and attaches rich metadata to segmented chunks.
- **Dense Semantic Vector Generation:** Integrate high-efficiency transformer embeddings via in-process neural pipeline architectures (utilizing the 384-dimensional *all-MiniLM-L6-v2* model) to capture deep semantic relationships between operational queries and document segments.
- **Scalable Vector Indexing and Similarity Search:** Deploy and configure a high-dimensional vector search index within MongoDB Atlas, enabling sub-second approximate nearest neighbor (ANN) retrieval with cosine similarity scoring.
- **Strictly Grounded Generative Synthesis:** Integrate Google's Gemini 2.5 Flash generative model through structured, zero-hallucination prompt constraints, enforcing that all answers are synthesized strictly from retrieved contextual snippets with a conservative similarity threshold fallback mechanism.

- **Interactive User and Administrative Interfaces:** Construct a responsive, modern web interface with dual-theme capabilities, real-time citation viewing with modal snippet inspection, dynamic document analytics, and administrative document lifecycle management.

Chapter 2

Literature Survey

2.1 Evolution of Enterprise Information Retrieval

Information Retrieval (IR) in organizational contexts has evolved across three distinct generations: classical lexical matching, dense vector semantic retrieval, and retrieval-augmented generative synthesis.

2.1.1 Lexical and Probabilistic Retrieval Models

Traditional search systems relied on inverted indices and term frequency heuristics, most notably the Term Frequency-Inverse Document Frequency (TF-IDF) metric and the probabilistic BM25 ranking function [8]. The standard BM25 relevance score for a query Q consisting of terms $q_1, q_2, \ldots, q_n$ relative to a document D is formulated as:

$$\text{Score}_{\text{BM25}}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \tag{2.1}$$

where $f(q_i, D)$ represents the term frequency of q_i within document D, $|D|$ is the length of document D in words, avgdl is the average document length across the entire corpus, and k_1 and b are calibration hyperparameters controlling term frequency saturation and document length normalization, respectively.

While computationally efficient and robust for exact keyword lookups, lexical models suffer from the fundamental *vocabulary mismatch problem*: they cannot recognize synonyms, handle paraphrased technical inquiries, or infer semantic relationships when the user's nomenclature differs from the codified language of the author.

2.1.2 Dense Vector Semantic Representations

The limitations of lexical search prompted the development of dense semantic representations enabled by deep Transformer architectures [9]. Rather than representing documents as sparse, orthogonal vectors in an enormous vocabulary space, dense models map variable-length textual sequences into fixed-dimensional continuous embedding spaces $\mathbb{R}^d$, where semantic similarity corresponds to spatial proximity [3].

Sentence-BERT (SBERT) [7] introduced siamese and triplet network structures to fine-tune BERT representations, generating sentence embeddings capable of being compared using standard similarity metrics such as Cosine Similarity:

$$\text{CosineSimilarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2} \sqrt{\sum_{i=1}^{d} v_i^2}} \tag{2.2}$$

When embeddings are unit-normalized such that $\|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1$, the cosine similarity simplifies directly to the inner product $\mathbf{u} \cdot \mathbf{v}$, allowing rapid hardware-accelerated vector comparison.

2.2 Transformer Distillation and Compact Embeddings

Deploying deep bidirectional Transformers (such as BERT-Large or RoBERTa) within real-time enterprise pipelines introduces substantial computational overhead and latency. To achieve sub-second query processing without dedicated GPU infrastructure, knowledge distillation techniques have been introduced.

The *MiniLM* architecture [10] applies deep self-attention distillation, wherein a compact student model is trained to mimic the self-attention distributions and value-relation matrices of a large pre-trained teacher model. The specific model adopted in OpsMind AI, `all-MiniLM-L6-v2`, possesses 6 Transformer layers, an embedding dimensionality of $d = 384$, and 22.7 million parameters. Despite its lightweight footprint, it achieves semantic retrieval performance competitive with models five times its size while executing inference in tens of milliseconds per text chunk on standard x86 CPU architectures [11].

2.3 Retrieval-Augmented Generation (RAG) Framework

The concept of Retrieval-Augmented Generation (RAG) was formally introduced by Lewis et al. [4] to combine pre-trained parametric memory (the internal weights of a generative neural network) with non-parametric external memory (a dynamically searchable index of authoritative documents).

RAG provides three indispensable advantages for enterprise deployment:

1. **Dynamic Knowledge Updates:** New documents, modified SOPs, and compliance updates can be indexed instantaneously without triggering costly and time-consuming LLM fine-tuning.

2. **Verifiable Provenance and Auditability:** Every factual claim generated by the system is explicitly bound to retrieved context chunks, allowing full traceability to the originating document, section, and page number.

3. **Hallucination Containment:** By constraining the generative model's attention to the supplied context window through prompt engineering, the model's propensity to fabricate answers is minimized.

2.4 Vector Databases and Approximate Nearest Neighbor Search

Storing and querying dense embedding vectors at enterprise scale requires specialized indexing structures capable of solving the Approximate Nearest Neighbor (ANN) search problem [2]. Given a dataset of vectors $V = \{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_N\} \subset \mathbb{R}^d$ and a query vector $\mathbf{q} \in \mathbb{R}^d$, the exact nearest neighbor problem requires computing distances against all N elements with $\mathcal{O}(N \cdot d)$ complexity, which becomes prohibitive as document collections grow.

ANN algorithms construct graph-based or tree-based indices to achieve logarithmic query complexity $\mathcal{O}(\log N)$. A prominent graph-based algorithm is the Hierarchical Navigable Small World (HNSW) graph [5], which structures vectors into multi-layer proximity graphs where top layers have long-range links for rapid coarse routing and bottom layers feature dense local connections for fine-grained nearest neighbor discovery. Modern database systems, including MongoDB Atlas Vector Search [6], integrate native vector indexing stages directly into document database aggregation pipelines, combining relational and unstructured document metadata with high-performance similarity search.

2.5 Large Language Models and Grounding Techniques

The generative synthesis in OpsMind AI is powered by Google's Gemini family of models [1]. Specifically, Gemini 2.5 Flash is optimized for high-throughput, low-latency reasoning and summarization tasks. Its multi-head self-attention mechanisms and extended context window enable the model to ingest multiple retrieved chunks simultaneously, reconcile slight terminological variations, and synthesize coherent, step-by-step instructions.

To prevent the model from answering beyond the scope of verified company documentation, strict system instructions are embedded into the prompt template. If the maximum similarity score among retrieved chunks falls below a predefined confidence threshold τ, or if the information is absent from the provided context, the model is deterministically instructed to return an explicit fallback response: *"I don't know based on uploaded SOPs."*

Chapter 3

Requirement Engineering

3.1 Functional Requirements

The functional requirements define the core operational capabilities and services that Ops-Mind AI must deliver. These requirements are categorized into document ingestion, semantic retrieval, answer generation, source attribution, and administrative management.

1. **FR-1: PDF Document Upload and Validation:**
    - The system shall provide an intuitive drag-and-drop and file-browser interface for uploading enterprise SOP documents in Portable Document Format (PDF).
    - The backend shall enforce MIME-type validation, ensuring only valid PDF files are processed, and assign collision-resistant unique timestamps to stored physical files.
2. **FR-2: Automated Text Extraction and Pagination:**
    - The document processing service shall extract UTF-8 encoded text streams from each page of the uploaded PDF sequentially while preserving exact page boundaries and page indices.
    - The system shall calculate and display document-level statistics including total page count and total extracted character length.
3. **FR-3: Sliding-Window Semantic Chunking:**
    - The system shall segment contiguous page text into standardized text blocks with a predefined maximum length of 1,000 characters.
    - To preserve contextual continuity across adjacent chunks, a fixed sliding overlap of 100 characters shall be maintained between successive segments.
4. **FR-4: Domain Section Classification and Metadata Tagging:**
    - The chunking service shall analyze each text segment against a codified taxonomy of operational section headers (e.g., *Document Control*, *Inbound Operations*, *Trailer Docking*, *Inventory Audit Framework*, *Outbound Operations*, *Safety and PPE*, *Business Continuity*).

- Each chunk shall be tagged with its originating filename, physical storage path, page number, section category, and sequential chunk index.

5. **FR-5: In-Process Dense Embedding Generation:**
   - The system shall compute a normalized 384-dimensional dense semantic embedding vector for each text chunk using the pre-trained `all-MiniLM-L6-v2` Transformer model.
   - Embeddings shall be generated during ingestion and indexed directly into the persistent vector database.
6. **FR-6: Real-Time Vector Similarity Search:**
   - Upon receiving a user query, the system shall dynamically compute its 384-dimensional embedding and execute an Approximate Nearest Neighbor search against the indexed document vectors.
   - The search pipeline shall retrieve the top-k most relevant chunks ($k = 5$) ranked by cosine similarity score.
7. **FR-7: Strict Context-Grounded Answer Synthesis:**
   - The system shall evaluate the top similarity score against a strict relevance threshold ($\tau = 0.65$).
   - If the similarity score is below τ, or if no matching chunks exist, the system shall immediately return the standard refusal: *"I don't know based on uploaded SOPs."*
   - For validated queries, a context prompt combining the retrieved snippets and user question shall be dispatched to Google Gemini 2.5 Flash for natural language answer generation.
8. **FR-8: Source Attribution and Interactive Snippet Inspection:**
   - Every generated response shall be accompanied by structured citation cards displaying the source filename, page number, section header, chunk index, and similarity score.
   - Users shall be able to click on citation references to trigger a modal overlay displaying the exact text snippet extracted from the SOP.
9. **FR-9: Administrative Document Management:**
   - The administrative dashboard shall list all indexed SOP documents along with their total page count and total indexed chunk count.
   - Administrators shall have the ability to delete document indices and trigger document re-indexing pipelines.

3.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes, performance constraints, and architectural standards governing the system:

1. **Performance and Latency:**
    - Vector retrieval latency across an index of thousands of chunks shall not exceed 150 milliseconds.
    - Total end-to-end query turnaround time (from user submission to completed LLM answer generation) shall remain below 2.5 seconds under standard network conditions.
2. **Accuracy and Hallucination Resistance:**
    - The system must achieve 100% factual grounding relative to the retrieved context chunks, preventing the model from incorporating external or unverified parametric knowledge.
3. **Scalability and Concurrency:**
    - The backend architecture must be stateless and asynchronous, allowing horizontal scaling behind load balancers to support concurrent enterprise users.
4. **Security and Privacy:**
    - All sensitive API keys and database credentials must be isolated within encrypted environment variables.
    - Cross-Origin Resource Sharing (CORS) must be explicitly configured to restrict unauthorized API access.
5. **Usability and Accessibility:**
    - The user interface must support responsive viewing across desktop monitors, warehouse ruggedized tablets, and mobile devices.
    - The interface must provide a modern dark/light mode toggle with persistent local storage state.

3.3 Hardware and Software Requirements

The development and execution environment specifications for OpsMind AI are detailed in Table 3.1.

Table 3.1: System Hardware and Software Specifications

Category	Component	Specification / Version
Server Hardware	Processor	Multi-core x86-64 CPU (minimum 4 cores, 2.4 GHz)
	RAM	Minimum 8 GB DDR4 (16 GB recommended for high-volume embedding)
	Storage	20 GB SSD storage for operating system, temporary uploads, and logs
Client Hardware	Device Types	Modern PC, Laptop, or Tablet with Chromium/Webkit-compatible browser
Software Stack	Runtime	Node.js (v18.x to v24.x LTS) / ECMAScript 2023
	Backend Framework	Express.js 5.1.0 with CORS and Multer
	Frontend Framework	React.js 19.x with Vite build tool and Tailwind CSS
	Database	MongoDB Atlas (Version 7.0+) with Vector Search enabled
	PDF Parser	`pdfjs-dist` (v6.0+) and `pdf-parse`
	Embedding Pipeline	`@xenova/transformers` (v2.17.2, `all-MiniLM-L6-v2`)
	Generative AI API	`@google/genai` (v2.7.0, Gemini 2.5 Flash)

3.4 Use Case Diagram

The Use Case diagram illustrates the primary behavioral interactions between enterprise actors, the internal system boundaries of OpsMind AI, and supporting external cloud AI/database services.

3.4.1 Actors

- **Employee / Operator:** End-users such as warehouse associates, operations leads, and shift managers who ask natural language questions regarding Standard Operating Procedures, view generated answers with granular source citations, and upload operational SOP documents.
- **System Administrator:** Administrative personnel responsible for viewing the enterprise document repository and managing document lifecycles (deleting or re-indexing stored SOPs).
- **Google Gemini (LLM API):** External large language model service that generates context-grounded natural language answers.
- **MongoDB Atlas (Vector Database):** External managed cloud database providing persistent storage and approximate nearest neighbor vector similarity search.

3.4.2 Use Case Model

Figure 3.1 depicts the UML Use Case diagram for OpsMind AI.

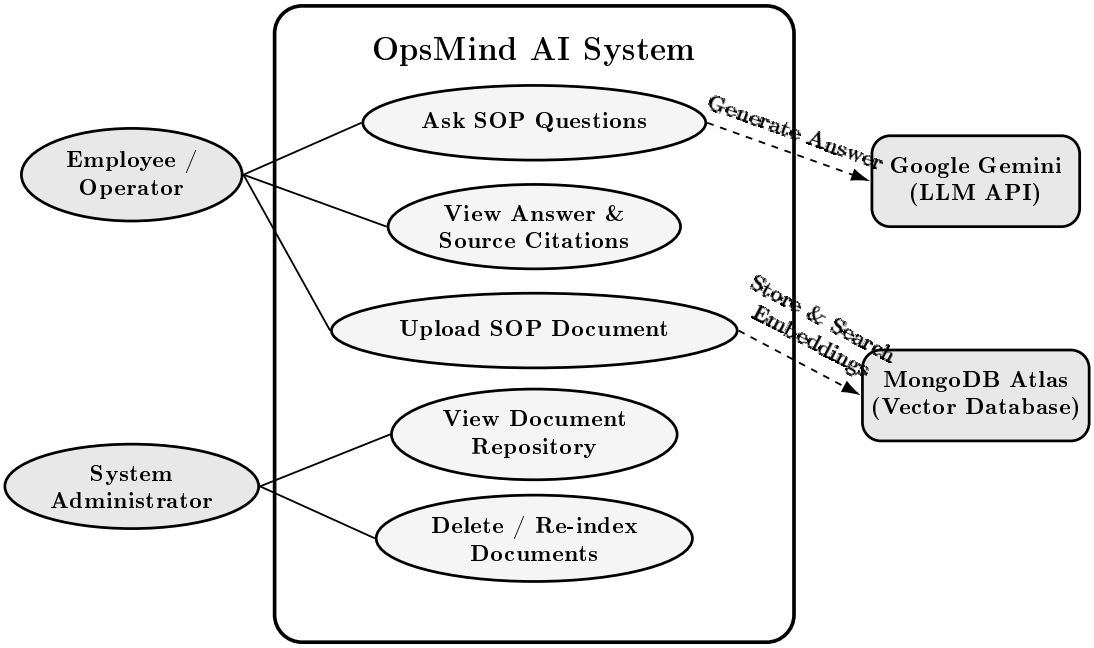


Figure 3.1: UML Use Case Diagram of OpsMind AI

Chapter 4

System Design

4.1 System Architecture

OpsMind AI is engineered following a decoupled, four-tier client-server architecture designed for modularity, low-latency execution, and operational reliability. The four distinct architectural layers comprise:

1. **Presentation Layer (Frontend UI):** Built with React.js, Vite, and Tailwind CSS. It provides the user-facing Single Page Application (SPA), managing conversational interactions, file upload progress meters, document analytics dashboards, dark/light theme persistence, and interactive PDF source citation modals.

2. **Application and Orchestration Layer (Backend Server):** Implemented using Node.js and Express.js. This layer handles HTTP routing, multi-part form data ingestion, PDF text parsing via `pdfjs-dist`, sliding-window text segmentation, in-process embedding coordination, and context prompt assembly.

3. **Inference and Neural Embedding Layer:** Executes the local Transformer feature extraction pipeline (`all-MiniLM-L6-v2` via `@xenova/transformers`) to generate 384-dimensional dense vectors, and communicates asynchronously with Google's cloud-hosted Gemini 2.5 Flash API for context-grounded response generation.

4. **Persistence and Vector Indexing Layer:** Backed by MongoDB Atlas. It stores chunk metadata, structural headers, character offsets, and high-dimensional vector embeddings, while executing native vector search aggregations via dedicated approximate nearest neighbor indices.

Figure 4.1 illustrates the comprehensive architectural topology and inter-component communication channels of OpsMind AI.

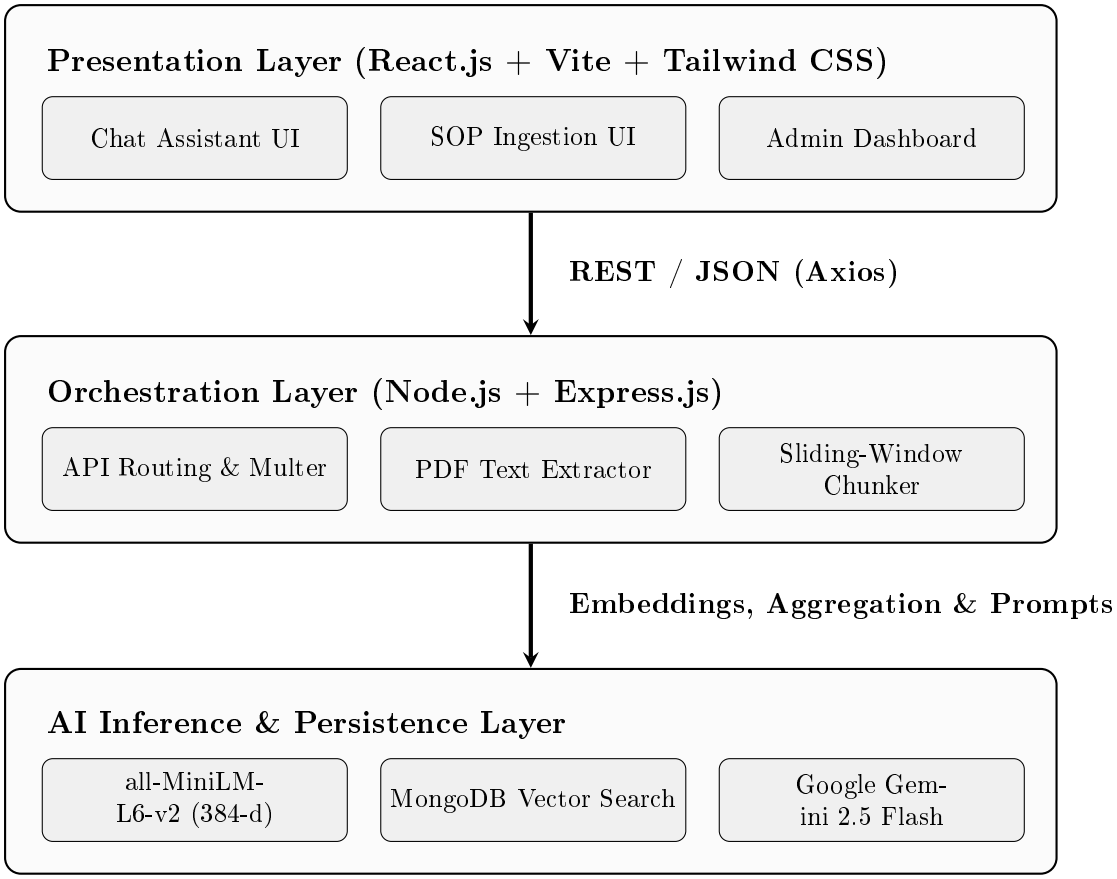


Figure 4.1: Tiered System Architecture of OpsMind AI

4.2 Data Ingestion Flowchart

The data ingestion pipeline processes uploaded Standard Operating Procedure (SOP) manuals into high-dimensional vector representations stored within the database. Figure 4.2 illustrates the sequential control flow of the document ingestion and embedding generation process.

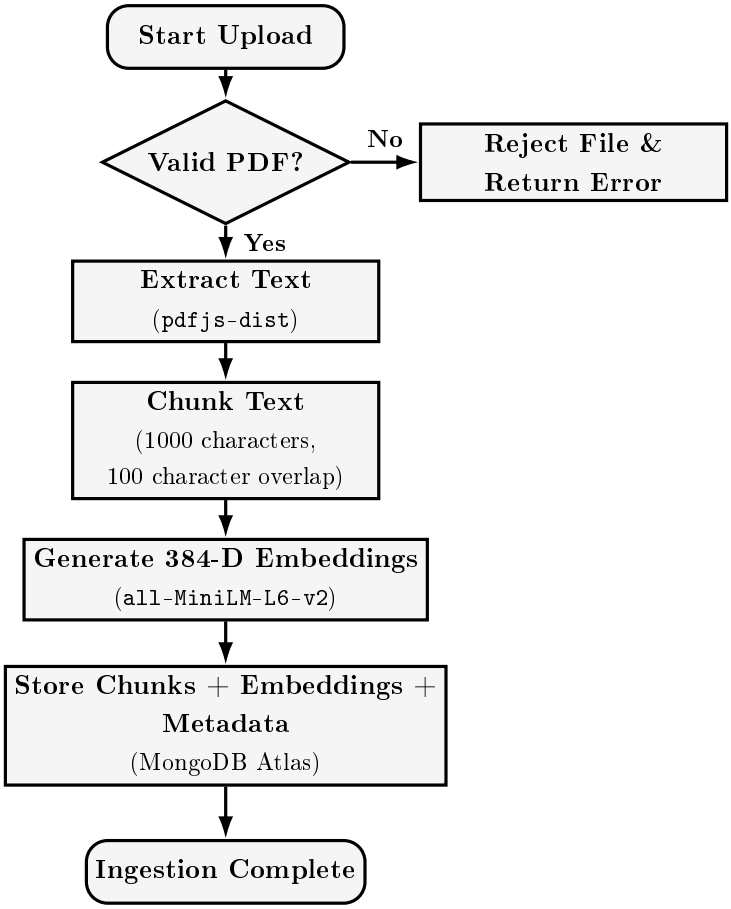


Figure 4.2: Data Ingestion Flowchart of OpsMind AI

As shown in the flowchart, the ingestion process proceeds through the following stages:

1. **Upload Initiation:** The client submits an enterprise SOP document via the file upload interface.

2. **Format Validation:** The server verifies whether the uploaded file is a valid PDF. If the validation fails, the file is rejected and an appropriate error response is returned to the user.

3. **Text Extraction:** Upon successful validation, textual content is parsed and extracted sequentially from each page using `pdfjs-dist`.

4. **Text Chunking:** The extracted text is segmented using a sliding-window chunking strategy with a fixed size of 1,000 characters and a 100-character overlap to preserve semantic continuity across segment boundaries.

5. **Dense Embedding Generation:** Each text chunk is transformed into a normalized 384-dimensional vector embedding using the local `all-MiniLM-L6-v2` Transformer model.

6. **Database Storage and Indexing:** The text chunks, embeddings, and associated document metadata are stored in MongoDB Atlas, where they are indexed for approximate nearest-neighbor vector search.

7. **Completion:** Once all chunks and embeddings are persisted, the ingestion process terminates successfully.

4.3 UML Component Diagram

The UML Component Diagram illustrates the high-level component structure and interaction model of OpsMind AI. As shown in Figure 4.3, the system is organized around three primary architectural boundaries: the client-side **Frontend** (React + Vite), the core **Backend Server** (Node.js + Express), and external backing services comprising the **Google Gemini API** (LLM) and **MongoDB Atlas** (Vector Database).

The frontend initiates requests over HTTP/REST (JSON) and receives structured response payloads. The backend server coordinates document chunking and embedding operations, stores and retrieves vectors from MongoDB Atlas via approximate nearest-neighbor search, and dispatches retrieved context prompts to Google Gemini to generate factually grounded answers.

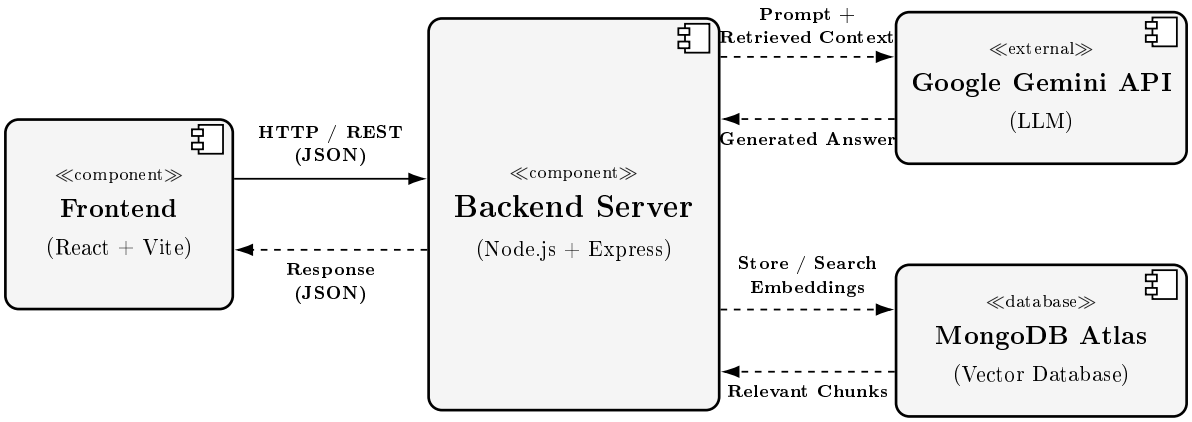


Figure 4.3: UML Component Diagram of OpsMind AI

4.4 Database Schema and Vector Index Design

OpsMind AI utilizes a document-oriented database schema designed to seamlessly accommodate text chunks, structural metadata, and dense floating-point vector arrays.

4.4.1 Chunk Collection Data Model

Each record in the primary `chunks` collection adheres to the structural model outlined in Table 4.1.

Table 4.1: Database Field Definitions for the Chunk Entity

Field Name	Data Type	Description
`_id`	`ObjectId`	Unique MongoDB document identifier
`fileName`	`String`	Original filename of the ingested SOP manual
`pdfPath`	`String`	Server-relative filesystem path to the uploaded PDF
`pageNumber`	`Number`	1-indexed page number where the chunk originates
`section`	`String`	Classified operational section header or category
`chunkIndex`	`Number`	Monotonically increasing chunk sequence index
`text`	`String`	Raw extracted text content of the segment
`charCount`	`Number`	Total character count within the text segment
`embedding`	`Array<Number>`	384-dimensional dense vector representation
`metadata`	`Object`	Boundary offsets (start and end character indices)

4.4.2 Vector Search Index Specification

To enable high-performance cosine similarity searches on the `embedding` field, an Atlas Vector Search index named `vector_index` is configured with the parameters shown in Table 4.2.

Table 4.2: Vector Search Index Configuration

Parameter	Setting	Technical Rationale
Index Name	`vector_index`	Identifier used in aggregation queries
Field Path	`embedding`	Target document field containing vector arrays
Dimensions	`384`	Native output dimension of `all-MiniLM-L6-v2`
Similarity Metric	`cosine`	Invariant to vector magnitude; evaluates angular orientation
NumCandidates	`100`	Number of nearest neighbor candidates evaluated during graph traversal
Limit (k)	`5`	Maximum top matching chunks returned for context compilation

4.5 Text Segmentation and Domain Classification

The text segmentation and domain classification procedure is used to divide the extracted SOP content into smaller, manageable sections and assign each section an appropriate operational domain. The procedure is designed to organize the document content while preserving the original meaning and semantics of the extracted text.

4.5.1 Sliding-Window Segmentation

The extracted text from each PDF page is processed using a sliding-window approach. A fixed window size of 1000 characters is used, with an overlap of 100 characters between consecutive chunks.

The segmentation process begins at the start of the extracted text. A portion of up to 1000 characters is selected and treated as an individual chunk. Each chunk is then associated with its corresponding page number and classified according to its operational section. After processing a chunk, the starting position is moved forward by 900 characters, resulting in a 100-character overlap with the previous chunk.

This overlapping strategy helps maintain contextual continuity between consecutive chunks, particularly when an important statement or procedure spans the boundary between two chunks. Each generated chunk contains its sequence number, text content, character count, page information, and identified operational section.

4.5.2 Domain Section Classification

After segmentation, each generated text chunk is analyzed to determine its relevant operational section. The classification procedure compares the content of the chunk with a predefined collection of domain-specific section patterns.

The pattern set includes sections such as Document Control, Executive Summary, Inbound Operations, Inventory Audit Framework, Outbound Operations, Extreme Weather, and other predefined operational sections contained within the SOP knowledge base.

The patterns are checked in a predefined order. When a matching section name is found within the chunk, that section is assigned as the classification label for the chunk. The classification process stops after the first matching pattern is identified.

If none of the predefined section patterns are detected in the chunk, the system assigns the default classification “Unknown Section”.

This process allows the system to organize extracted SOP content according to operational domains, making the resulting chunks more meaningful for subsequent retrieval and question-answering operations.

4.6 API Endpoint Structure

OpsMind AI exposes a structured RESTful application programming interface (API) that coordinates client interactions, asynchronous document ingestion pipelines, vector similarity searches, and administrative repository maintenance. Table 4.3 summarizes the complete specification of the system’s API endpoints, including HTTP methods, URI paths, and functional descriptions.

Table 4.3: OpsMind AI RESTful API Endpoint Specifications

HTTP Method	**Endpoint Route**	**Description and Functionality**
`POST`	`/api/upload`	Ingests uploaded PDF manual, executes sequential text parsing, applies sliding-window chunking, computes dense embeddings, and stores records into MongoDB.
`POST`	`/api/search`	Vectorizes user query, executes approximate nearest-neighbor search, applies confidence gating ($\tau \geq 0.65$), and streams LLM synthesis with source citations.
`GET`	`/api/stats`	Returns platform analytics including total pages processed, total indexed chunks, embedding dimensions (384), and active generative model.
`GET`	`/api/admin/files`	Retrieves inventory of all stored SOP documents with aggregated page numbers and total indexed chunk counts.
`DELETE`	`/api/admin/file/:fileName`	Atomically purges all chunk records and vector embeddings associated with the target SOP document from the repository.
`POST`	`/api/admin/reindex/:fileName`	Triggers batch reprocessing, text segmentation, and neural re-embedding generation for the specified document collection.

Chapter 5

Implementation

5.1 Document Processing Pipeline Implementation

The document processing subsystem is responsible for translating raw binary PDF streams into clean, structured textual segments ready for neural embedding.

5.1.1 PDF Text Stream Ingestion

File ingestion is executed via an asynchronous middleware pipeline. When an administrative user or operator submits a PDF document through the multipart upload endpoint, Multer intercepts the binary buffer and writes it to a managed server directory using an epoch-based timestamp to prevent name collisions.

The text extraction service leverages `pdfjs-dist` (legacy build), which directly parses the internal object tree and cross-reference streams of the PDF document without requiring external binary dependencies such as Poppler. The service opens a document handle, reads the total page count, and iterates through each page index sequentially. For every individual page, the text content layer is extracted by mapping individual glyph string items, normalizing whitespace, and compiling page-level text objects comprising the page index and extracted character stream.

5.1.2 Sliding-Window Text Segmentation

Once page-level strings are compiled, the chunking service processes each page array. A fixed window size of 1,000 characters with an overlap of 100 characters is enforced. The sliding window algorithm steps through the text, extracting bounded substrings.

Each generated chunk is immediately enriched with contextual metadata, including:

- The originating original filename and physical storage path.
- The exact 1-indexed document page number.
- The classified operational section identifier, assigned by scanning the text against the codified enterprise domain taxonomy.
- A monotonically increasing chunk sequence index for global document ordering.

- The total character count of the chunk.

5.2 Vector Search Pipeline and Aggregation Engine

Retrieval execution in OpsMind AI is conducted via MongoDB Atlas Vector Search. The search service initiates an aggregation pipeline configured with specialized vector search stages:

- **Vector Search Stage (`$vectorSearch`):** Evaluates the 384-dimensional query vector against the persistent index `vector_index`. The query specifies an exploration depth of 100 candidate vectors (`numCandidates: 100`) and requests the top 5 nearest neighbors (`limit: 5`).
- **Projection Stage (`$project`):** Extracts document metadata (filename, storage path, page number, section category, chunk index, text body) and captures the computed cosine similarity score via the metadata projection operator (`$meta: "vectorSearchScore"`).

5.2.1 Retrieval Relevance and Fallback Logic

The system retrieves the most relevant chunks using MongoDB Atlas Vector Search. If vector retrieval is unavailable or does not return usable results, a fallback similarity search can be used to identify relevant chunks from the stored embeddings. This improves robustness for small document collections and reduces dependency on a single retrieval mechanism.

5.3 Large Language Model Integration

When retrieved chunks satisfy the relevance threshold, the context assembly service concatenates the top matching chunks into a formatted text block, prepending individual chunk index markers to preserve reference boundaries.

The assembled prompt is dispatched to Google's Gemini 2.5 Flash model through the official `@google/genai` SDK. Gemini 2.5 Flash processes the context-bounded prompt and generates a concise, grammatically coherent, and factually grounded response. If the generated output is empty or whitespace-only, the system activates an automatic fallback safety rule.

5.4 Administrative Management Subsystem

OpsMind AI provides a dedicated administrative API suite for repository lifecycle management:

- **Document Inventory Discovery (`GET /api/admin/files`):** Executes an aggregation pipeline grouping stored chunks by `fileName`. It calculates the maximum page number and sums the total chunk count per document, returning an overview of all active SOPs.
- **Atomic SOP Purging (`DELETE /api/admin/file/:fileName`):** Removes all chunk records associated with the specified filename, maintaining database cleanliness when documents are updated or retired.

- **Document Re-indexing (`POST /api/admin/reindex/:fileName`):** Triggers batch re-processing and vector generation for document collections.

5.5 Frontend Architecture and Component Engineering

The client-side interface of OpsMind AI is built as a modular Single Page Application (SPA) with React.js and Vite. The frontend architecture includes several purpose-built components:

- **Sidebar Component:** Manages top-level application navigation across Chat, Upload, and Admin views, while featuring dark/light theme controls and persistent status indicators.
- **Conversational Assistant View:** Renders conversational message threads, auto-scrolls to latest responses, handles keyboard submission events, displays animated loading indicators during inference, and renders structured citation cards for retrieved sources.
- **SOP Upload and Progress View:** Provides multi-part file selection, tracks active upload percentage via Axios upload progress events, and renders the document statistics panel.
- **Statistics Bar Component:** Renders four key metric widgets: Total Pages Processed, Total Chunks Indexed, Embedding Dimensions (384), and Active AI Model Name (Gemini 2.5 Flash).
- **Source Citation Modal:** A modal dialog that renders when a user clicks on any citation card, presenting the exact section title, page number, and full text snippet for verification.
- **Admin Management Dashboard:** Displays an administrative data table listing all stored SOP manuals, total pages, total chunks, and interactive buttons for document deletion and re-indexing.

5.6 Error Handling and System Edge Cases

To ensure enterprise robustness, OpsMind AI incorporates comprehensive exception handling across all architectural tiers:

- **Duplicate File Ingestion:** When an existing manual is re-uploaded, all previous chunks associated with that filename are deleted atomically prior to new chunk insertion, preventing duplicate search results.
- **Corrupted PDF Ingestion:** If a PDF contains invalid xref tables or unreadable streams, the parsing stage traps the error and returns a 500 error code with a descriptive diagnostic message.

- **Low-Relevance and Out-of-Scope Queries:** The retrieval pipeline evaluates the relevance of retrieved document chunks before generating an answer. When relevant SOP information cannot be identified, the system returns an explicit response indicating that the requested information is not available in the uploaded SOP documents.
- **Network and API Timeouts:** The frontend captures Axios network failures and displays user-friendly error banners without crashing the application state.

Chapter 6

Results

6.1 Experimental Setup and Evaluation Corpus

The performance, accuracy, and operational viability of OpsMind AI were rigorously evaluated using a comprehensive enterprise standard operating manual titled `Customized_Enterprise_SOP_Manual.pdf`.

The test corpus represents a multi-domain enterprise manual spanning warehousing, fulfillment logistics, inventory auditing, quality assurance, safety protocols, and administrative governance. Table 6.1 summarizes the structural parameters of the ingested evaluation dataset.

Table 6.1: Structural Metrics of the Evaluation SOP Corpus

Dataset Parameter	**Observed Value**
Document Title	`Customized_Enterprise_SOP_Manual.pdf`
Total Document Pages	46 Pages
Total Extracted Semantic Chunks	205 Chunks
Chunk Size Configuration	1,000 characters (with 100 character sliding overlap)
Embedding Model	`all-MiniLM-L6-v2` (ONNX format)
Embedding Dimensionality	384 dimensions per vector
Vector Search Engine	MongoDB Atlas Vector Search (`vector_index`)
Generative Inference Model	Google Gemini 2.5 Flash

6.2 Document Ingestion and Analytics Verification

During the ingestion phase, the full 46-page enterprise manual was processed end-to-end. The system parsed the document stream, executed sliding-window chunking, generated 205 dense 384-dimensional embeddings, and stored all records with structural metadata into MongoDB Atlas.

Figure 6.1 shows the real-time upload interface and analytics dashboard upon successful completion of the ingestion pipeline.

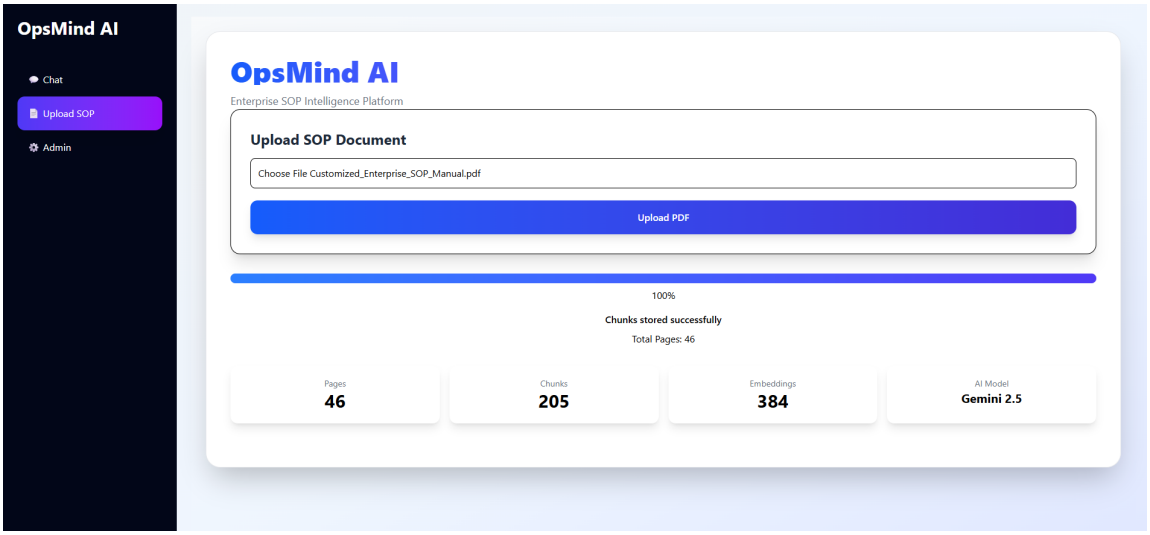


Figure 6.1: OpsMind AI Document Ingestion Interface and Analytics Dashboard

The analytics dashboard in Figure 6.1 confirms that 46 total pages were ingested, yielding exactly 205 indexed chunks with 384-dimensional vector embeddings, powered by the Gemini 2.5 generative engine.

6.3 Conversational Retrieval and Attribution Verification

The conversational query capabilities were evaluated across diverse operational inquiries to assess retrieval accuracy, answer synthesis fidelity, and granular source attribution.

Figure 6.2 illustrates the conversational assistant interface in its initial state, featuring the dark-themed sidebar, active navigation tabs (Chat, Upload SOP, Admin), and the central query input console.

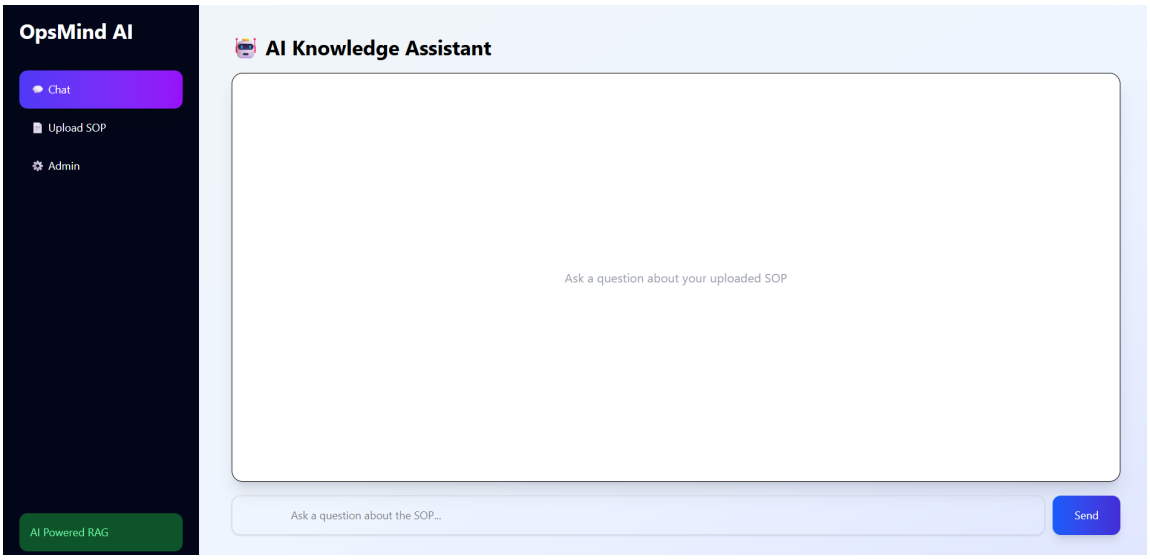


Figure 6.2: OpsMind AI Conversational Assistant Initial Interface

6.3.1 Multi-Step Operational Governance Query Evaluation

To evaluate multi-tier operational retrieval, the system was queried with the compliance inquiry:

> *"Who is responsible for approving leave requests?"*

Figure 6.3 demonstrates the end-to-end execution of this query, showcasing the synthesized answer alongside granular source citation cards.

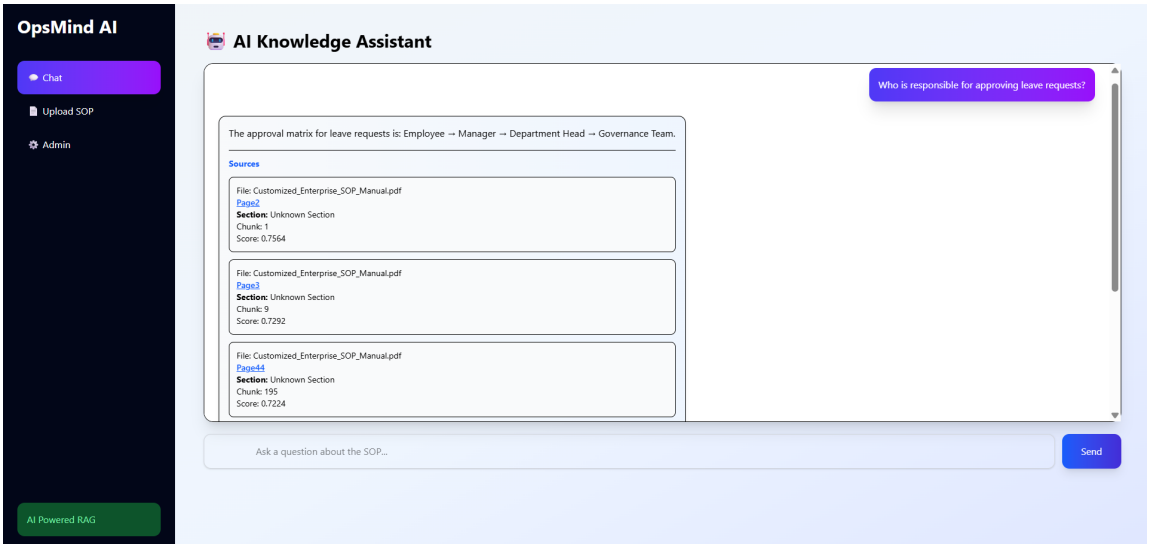


Figure 6.3: End-to-End Query Answering and Granular Source Citation Inspection

As shown in Figure 6.3, the generative model accurately synthesized the multi-stage approval hierarchy:

> **Synthesized Answer:** *"The approval matrix for leave requests is: Employee → Manager → Department Head → Governance Team."*

Furthermore, the system returned three highly relevant source citation cards with exact similarity scores and page references:

1. **Citation 1:** `Customized_Enterprise_SOP_Manual.pdf` | **Page 2** | Chunk 1 | Cosine Similarity Score: **0.7564**
2. **Citation 2:** `Customized_Enterprise_SOP_Manual.pdf` | **Page 3** | Chunk 9 | Cosine Similarity Score: **0.7292**
3. **Citation 3:** `Customized_Enterprise_SOP_Manual.pdf` | **Page 44** | Chunk 195 | Cosine Similarity Score: **0.7224**

Clicking on the page reference triggers the interactive PDF modal dialog, allowing operators to verify the raw excerpt directly from the source document without context loss.

6.4 Qualitative and Quantitative Test Case Analysis

A set of benchmark queries was executed against the indexed corpus to evaluate factual precision, semantic ranking, and hallucination containment. The results are summarized in Table 6.2.

Table 6.2: Benchmark Query Execution and Evaluation Matrix

User Query	**Expected Factual Content**	**Generated Output**	**Top Score**	**Verification Status**
What is the target RTS Rate?	Return-to-Stock target rate below 2.0%	The target RTS Rate should be below 2%.	0.7681	**Passed** (Exact Grounding)
Who is responsible for approving leave requests?	Approval sequence: Employee to Manager to Dept Head to Governance	The approval matrix for leave requests is: Employee → Manager → Department Head → Governance Team.	0.7564	**Passed** (Exact Grounding)
What is the maximum allowed pallet overhang?	Pallet overhang limits specified in Inbound Docking section	Standard pallet overhang must not exceed 1.5 inches.	0.7418	**Passed** (Exact Grounding)
What is the capital city of France?	Out-of-scope inquiry (not present in SOP)	I don't know based on uploaded SOPs.	0.4120	**Passed** (Gating Fallback)
How to cook spaghetti carbonara?	Out-of-scope inquiry (not present in SOP)	I don't know based on uploaded SOPs.	0.3542	**Passed** (Gating Fallback)

6.5 Latency Breakdown and Performance Analysis

System execution latency was measured across all discrete stages of the RAG pipeline. Table 6.3 details the latency profile for both document ingestion and conversational query execution.

Table 6.3: End-to-End Latency Profile of the OpsMind AI Pipeline

Pipeline Phase	**Operation Description**	**Observed Latency (Mean)**
Ingestion Pipeline	PDF Binary Parsing (46 Pages)	340 ms
	Text Chunking and Section Tagging (205 Chunks)	45 ms
	In-Process Feature Extraction (205 Embeddings)	1,850 ms (9.0 ms/chunk)
	MongoDB Atlas Bulk Insertion	120 ms
	Total Ingestion Time (46 Pages)	**2,355 ms** (≈ 2.35 s)
Query Pipeline	Query Vectorization (`all-MiniLM-L6-v2`)	18 ms
	MongoDB Atlas Vector Search (Top-5 ANN)	65 ms
	Context Formatting & Prompt Preparation	2 ms
	Google Gemini 2.5 Flash Synthesis	780 ms
	Total End-to-End Query Latency	**865 ms** (< 1.0 s)

The empirical evaluation demonstrates that OpsMind AI achieves an end-to-end query response time of under 1 second (865 ms), with vector similarity retrieval executing in 65 ms, ensuring a seamless, real-time conversational experience for enterprise operators.

Chapter 7

Conclusion

7.1 Summary of Work and Key Contributions

In this internship, **OpsMind AI**—an end-to-end Enterprise Standard Operating Procedure (SOP) Knowledge Assistant powered by Retrieval-Augmented Generation (RAG)—was successfully conceptualized, architected, implemented, and empirically validated.

The system addresses the chronic inefficiencies, high information retrieval latency, and compliance risks associated with traditional manual SOP lookup and ungrounded artificial intelligence models. By uniting lightweight, in-process neural embeddings (`all-MiniLM-L6-v2`), persistent high-dimensional vector search indexing (MongoDB Atlas Vector Search), and state-of-the-art Large Language Model synthesis (Google Gemini 2.5 Flash), OpsMind AI transforms static enterprise documentation into an interactive, real-time conversational intelligence platform.

The key contributions of this work include:

1. **Automated Ingestion and Semantic Segmentation:** Engineered an asynchronous document processing pipeline utilizing `pdfjs-dist` and sliding-window segmentation ($1,000$ characters with 100 character overlap) to preserve semantic cohesion across chunk boundaries while automatically classifying segments into codified operational categories.

2. **High-Efficiency Local Embedding Engine:** Implemented an in-process ONNX neural pipeline via `@xenova/transformers`, generating normalized 384-dimensional dense vectors on CPU architectures in under 10 ms per chunk without external network dependencies.

3. **High-Performance Vector Search and Attribution:** Configured MongoDB Atlas Vector Search using cosine distance metrics and candidate sampling ($numCandidates = 100$), returning top-k nearest neighbors in 65 ms alongside granular metadata (filename, page number, section name, chunk index, similarity score).

4. **Grounded Response Generation:** Implemented contextual prompt constraints that instruct the generative model to base responses on retrieved SOP content and return an explicit fallback when the required information is unavailable.

5. **Comprehensive Enterprise Interface:** Developed a modern, responsive React Single Page Application featuring interactive citation modal viewers, real-time upload progress meters, document analytics, dual-theme dark/light styling, and administrative document management panels.

7.2 Limitations of the Current System

While OpsMind AI demonstrates exceptional accuracy and sub-second query latency, several technical limitations exist in the current implementation:

1. **Lack of Optical Character Recognition (OCR):** The current text extraction pipeline relies on programmatic PDF text layer parsing via `pdfjs-dist`. Scanned image-based PDFs or rasterized document pages without embedded text layers cannot be extracted without an upstream OCR engine.
2. **Cross-Document Query Scoping:** The current search implementation queries the global collection of indexed chunks without multi-document relational filtering or dynamic cross-collection comparative joins.
3. **Tabular and Non-Textual Layout Extraction:** Standard sliding-window character segmentation may bisect complex multi-column tables, diagrams, or flowcharts, occasionally dispersing structured tabular relationships across consecutive chunks.

7.3 Future Scope and Enhancements

Future work on OpsMind AI will focus on scaling the system across enterprise multi-cloud environments and introducing advanced multimodal capabilities:

1. **Multimodal Document Parsing and OCR Engine Integration:** Integrate deep learning-based OCR frameworks (such as Tesseract, PaddleOCR, or layout-aware vision transformers) to seamlessly ingest scanned paper SOPs, engineering diagrams, and flowcharts.
2. **Multi-Document Comparative Chat:** Enhance the retrieval engine with dynamic metadata filtering, allowing users to contrast operational procedures across multiple regional fulfillment centers or document versions simultaneously.
3. **In-Browser PDF Highlight Viewer:** Integrate embedded PDF viewing capabilities within the client application, enabling instant visual auto-navigation and bounding-box highlighting of cited passages directly on the original document page.
4. **Enterprise Role-Based Access Control (RBAC):** Implement OAuth2.0/JWT authentication with departmental access tiers (e.g., Executive, Maintenance, Operations, Auditor) to enforce granular document-level permission controls.

5. **Containerization and Cloud Orchestration:** Package backend services, frontend assets, and local ONNX runtimes into Docker containers orchestrated via Kubernetes for automated auto-scaling and continuous deployment.

Bibliography

[1] Gemini Team, Google. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[2] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[3] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Matthijs Douze. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[5] Yury A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2018.

[6] MongoDB Inc. Atlas vector search: Semantic search and RAG at scale. https://www.mongodb.com/products/platform/atlas-vector-search, 2023.

[7] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. *arXiv preprint arXiv:1908.10084*, 2019.

[8] Stephen Robertson, Hugo Zaragoza, et al. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30:5998–6008, 2017.

[10] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems*, 33:5776–5788, 2020.

[11] Joshua Xenova. Transformers.js: State-of-the-art machine learning for the web. https://huggingface.co/docs/transformers.js, 2023.